



Shifa Tameer-e-Millat University

Scope Document

for

ProfessorOS

AI-Powered Academic Assessment Platform

with Explainable Grading and Intelligent Proctoring

Version 1.0

Submitted By

Muhammad Yasin Asif BSSE-23S-0036

Muhammad Fahim BSSE-23S-0033

Rayyan Shahid BSSE-23S-0037

Supervisor: Mam Habib-un-Nisa

Co-Supervisor: Mr. Muhammad Shaban Qabil

Bachelor of Science in Software Engineering (2023–2027)

Table of Contents

Table of Contents

Abstract.....	4
1. Introduction	4
2. Problem Statement	5
3. Problem Solution/Objectives of the Proposed System	5
3.1 Objectives	6
4. Related System Analysis/Literature Review.....	7
5. Vision Statement.....	8
6. Scope	8
7. Modules	9
Flutter Web App (Professor and Student Portals).....	9
Module 1: Authentication and User Management.....	9
Module 2: Course and Assignment Management.....	9
Module 3: AI Grading and Evaluation.....	10
Module 4: Explainable AI (XAI) Processing.....	10
Module 5: Student Submission Portal	10
Module 6: Analytics Dashboard.....	10
Flutter Mobile App.....	11
Module 7: Smart Proctoring	11
Module 8: Code Editor and Compiler (Mobile-accessible).....	11
Module 9: AI Teaching Assistant.....	11
8. Non-Functional Requirements.....	11
9. System Limitations/Constraints	12
10. Data Gathering Approach.....	12
11. Tools and Technologies	13
12. Project Stakeholders and Roles.....	14
13. Module Based Work Division.....	15
14. WBS and Gantt Chart	16
15. Mockups.....	17
Mockup 1: Course and Assignment Rubric Builder.....	17
Mockup 2: AI Grading Review Panel.....	18

Mockup 3: Analytics Dashboard	19
Mockup 4: Student Grade Feedback Viewer with XAI	20
Mockup 5: Integrated Code Editor and Compiler	21
Mockup 6: Mobile Smart Proctoring Focus Mode	22
16. References	24
17. Plagiarism Report Attach the Plagiarism report of your project scope document from library staff of Turnitin tool (http://turnitin.com).....	25

Abstract

ProfessorOS is a platform developed to address a critical inefficiency in Pakistani higher education: professors dedicate 12–18 hours every week to manual grading, time that would otherwise be spent on teaching and student development. Most institutions still rely on paper marking or basic digital uploads with zero automation, while existing commercial tools are either too expensive or fail to address the real pain points of local higher education. The core problems are grading delays that leave students waiting one to two weeks for feedback, inconsistent scores introduced when teaching assistants apply rubrics subjectively. No single platform currently handles essays, code submissions, and proctored examinations together without requiring professors to manage multiple disconnected tools. The identified gap is the complete absence of a locally deployable, open-source system that delivers explainable AI grading combined with lightweight mobile proctoring, free of any paid cloud dependency. The platform aims to reduce grading time by at least 75 percent, deliver criterion-level feedback within 60 seconds and provide transparent LIME/SHAP explanations so both professors and students understand every grading decision. ProfessorOS provides Pakistani universities with a practical, self-contained assessment ecosystem aligned to their budget and infrastructure realities, while preserving the professor’s full academic authority.

1. Introduction

Higher education in Pakistan has expanded rapidly over the past two decades, with more than 1.6 million students enrolled across over 200 chartered universities as of 2023 [8]. This growth has placed severe pressure on assessment infrastructure that has remained largely unchanged, with most institutions still relying on manual paper marking or basic file-upload systems with no grading automation. The practical consequence for professors is a grading burden of 12–18 hours per assignment cycle, time taken directly from instruction and student support. The specific pain points are consistent:

feedback delays of one to two weeks, scoring inconsistencies introduced by teaching assistants who interpret rubrics differently, and the absence of any in-platform code execution environment. Existing commercial platforms such as Gradescope and Turnitin address these problems only partially and in fragmented, proprietary, and cost-prohibitive ways. No locally deployable, open-source solution currently exists that combines explainable AI grading, semantic plagiarism detection, integrated code execution, and lightweight mobile proctoring in a single unified system. ProfessorOS is proposed to fill this gap by providing an AI-powered academic assessment platform that runs entirely on institutional hardware, requires no recurring cloud costs, and keeps the professor in full academic control through a human-in-the-loop architecture. This proposal serves as the formal reference document for the development cycle spanning from April 2026 through February 2027.

2. Problem Statement

Professors at Pakistani universities are buried under a workload that no human can handle fairly week after week. A typical professor teaching two courses with 180 students each must grade roughly 360 submissions per assignment cycle. In one live shadowing session we did at STMU, we timed it: seven minutes per written response. That adds up to more than 21 hours of pure marking for a single cycle. When teaching assistants help, the scores swing by as much as three marks out of ten because each person interprets the differently. Students sit there for one to two weeks with no feedback, so the same conceptual mistakes keep repeating. Programming courses make it worse no one has an in-platform compiler, so professors waste even more time downloading files and running tests manually. We kept hearing the same complaint: “We need something that actually works here, not some fancy foreign system we cannot afford.” These problems are connected, and fixing only one of them will not solve the mess. That is exactly why we are building ProfessorOS.

3. Problem Solution/Objectives of the Proposed System

ProfessorOS routes every submission through a locally running Llama 3.1 8B model via Ollama so grading starts the moment a student hits submit. The model scores against the professor's own rubric and spits out natural-language feedback for each criterion within about 60 seconds. A professor can override any score in one click and add a justification the final decision always stays human. Because the same rubric and model logic apply to every submission, inter-rater variation disappears. Students see colour-coded LIME and SHAP explanations so they finally understand why they lost marks instead of guessing. For code assignments we embedded re_editor and route everything through Judge0 so compilation and test-case execution happen instantly inside the Flutter app. The Flutter mobile app adds lightweight focus-mode proctoring that only watches whether the student switches apps no camera, no creepy surveillance. An AI Teaching Assistant built with LangChain answers course-specific questions 24/7 using the professor's own uploaded materials. Everything is containerised with Docker Compose and deploys in one command on university servers. We designed the whole pipeline so professors keep full academic authority while the repetitive grind is handled automatically. The system even gives real-time dashboards so course coordinators can spot at-risk students early.

3.1 Objectives

BO-1: Reduce professor grading time per assignment cycle by a minimum of 75 percent through automated AI evaluation of MCQ, short answer, essay, and programming submissions.

BO-2: Deliver criterion-level natural language feedback to students within 60 seconds of assignment submission, replacing the existing 7 to 14 day feedback delay.

BO-3: Achieve consistent grading output with inter-rater reliability exceeding 85 percent across all submission types through rubric-anchored evaluation.

BO-4: Implement an Explainable AI layer using LIME and SHAP frameworks to generate token-level and criterion-level explanations for every grading decision visible to both professors and students.

BO-5: Deliver a multi-language code execution environment supporting 12 or more programming languages with automated test case validation and AI-based code quality scoring.

BO-6: Reduce single-device examination cheating by a minimum of 60 percent through the Smart Proctoring Focus Mode that monitors application focus without invasive camera surveillance.

BO-8: Provide a 24/7 AI Teaching Assistant capable of answering course-specific questions using course materials uploaded by the professor as context.

BO-9: Enable data-driven course management through real-time dashboards tracking grade distributions, rubric criterion performance, and at-risk student identification.

BO-10: Deliver a fully containerized, cross-platform system deployable on institutional hardware using Docker Compose with a CI/CD pipeline through GitHub Actions.

4. Related System Analysis/Literature Review

Gradescope lets instructors batch similar answers but offers zero explainability and no mobile proctoring or code compiler. Google Classroom is basically a file dropbox with zero automation or analytics. Moodle handles basic MCQs but collapses on essays, code, and proctoring.

Table 1: Related System Analysis with Proposed Project Solution

Application Name	Weakness	Proposed Project Solution
Gradescope	No explainability, no mobile proctoring, no code compiler, paid-only	LIME/SHAP explainability, Flutter focus-mode proctoring, Judge0 compiler, locally deployable at no per-use cost

Google Classroom	Zero automated grading, no analytics, no proctoring	End-to-end AI grading pipeline covering all submission types with analytics and proctoring
Moodle	No AI grading engine, no explainability, no mobile proctoring	Unified platform deployed in one command via Docker Compose with full AI, XAI, and proctoring

5. Vision Statement

For university professors, teaching assistants, and students at Pakistani higher education institutions who are burdened by slow, inconsistent, and opaque assessment workflows that delay feedback and provide students with insufficient guidance to improve their work, the ProfessorOS platform is an AI-powered, cross-platform academic assessment ecosystem. It automates grading with full criterion-level explainability, proctors examinations non-invasively on mobile devices, compiles and evaluates programming submissions in real time, and delivers structured student feedback within 60 seconds of submission all governed by a human-in-the-loop model that preserves the educator's final academic authority. Unlike Gradescope or Google Classroom which address these problems in fragmented, proprietary, and cost-prohibitive silos ProfessorOS unifies every dimension of the assessment lifecycle into a single transparent, open-source, and locally deployable platform. This enables Pakistani universities to deliver consistent and timely academic assessment without dependence on expensive foreign commercial services.

6. Scope

ProfessorOS covers the complete assessment lifecycle from course creation through submission, AI grading, human review, and analytics. The system includes a Professor Web Portal, Student Web Portal, Flutter Mobile App, University Admin Dashboard, and a dedicated AI/ML Engine. Authentication uses institutional email with JWT and role-based access across all portals. Professors can create courses, build detailed rubrics, generate questions from uploaded materials, and assign batches to teaching assistants. Every submission : text, code, MCQ, or file goes through

the Llama 3.1 8B grading pipeline with LIME and SHAP explanations attached. Students submit via rich editor or mobile, view their colour-coded feedback, and can raise disputes. The code editor uses re_editor with Judge0 compilation supporting 12 languages and hidden test cases. Smart Proctoring on mobile only watches app focus and logs violations without any camera. Real-time dashboards show grade distributions, at-risk students, and queue performance. An AI Teaching Assistant answers course questions using professor-uploaded materials via LangChain. The entire stack is Dockerised and includes OpenAPI documentation plus GitHub Actions CI/CD. We deliberately left out video proctoring, facial recognition, handwritten scanning, LLM fine-tuning, ERP integration, and offline-first operation beyond cached grades because those would push us outside our timeline and hardware constraints at STMU. The result is a focused, practical platform that actually fits Pakistani university realities.

7. Modules

Flutter Web App (Professor and Student Portals)

Module 1: Authentication and User Management

- The system shall allow users to register and log in using institutional email with JWT-based session management across Professor, Student, and Admin portals.
- The system shall support biometric authentication for subsequent mobile logins and enforce Argon2 password hashing with session invalidation on password update.
- The system shall enable administrators to create and manage professor, student, and TA accounts with role-based access control and bulk CSV import for student records.

Module 2: Course and Assignment Management

- The user shall be able to create, edit, archive, and delete courses, and enroll students individually or through bulk CSV upload.
- The user shall be able to build structured grading rubrics with named criteria, level descriptors, and point weight sliders.

- The system shall generate multiple choice and short answer questions from uploaded course materials using the AI engine.

Module 3: AI Grading and Evaluation

- The system shall receive grading tasks from the Celery queue and parse model output into structured criterion scores and natural-language feedback.
- The user shall be able to view AI-assigned scores alongside each submission and approve, flag, or override any score in a single click.

Module 4: Explainable AI (XAI) Processing

- The system shall apply the LIME text explainer to generate token-level influence scores rendered as highlight spans on the submission text.
- The system shall apply the SHAP partition explainer to generate criterion-level attributions displayed as colour-coded overlays.

Module 5: Student Submission Portal

- The user shall be able to view active assignments with deadlines and receive in-platform notifications for new assignments and graded feedback.
- The user shall be able to submit text, files, or MCQ answers and view criterion-level feedback with XAI highlight overlays upon grading completion.

Module 6: Analytics Dashboard

- The system shall display grade distribution histograms, rubric criterion radar charts, at-risk student lists, and time-series cohort performance data on the analytics dashboard.

Flutter Mobile App

Module 7: Smart Proctoring

- The system shall activate focus mode during examinations and display the violation count with a return prompt whenever the student switches applications.
- The system shall log all focus violations via the AppLifecycleState API and transmit real-time violation updates to the professor portal via WebSocket.

Module 8: Code Editor and Compiler (Mobile-accessible)

- The user shall be able to write and execute code in re_editor with Judge0-powered compilation and view test-case results inline.

Module 9: AI Teaching Assistant

- The user shall be able to submit natural language questions and receive responses grounded in the professor-uploaded course materials via the LangChain RAG pipeline.

8. Non-Functional Requirements

NFR-1 (Security): The system shall authenticate all users via JSON Web Tokens (JWT) with a configurable expiry period. The system shall store all passwords using Argon2 hashing. The system shall enforce role-based access control so that professors, students, teaching assistants, and administrators can only access the resources their role permits.

NFR-2 (Performance): The system shall deliver AI-generated criterion-level feedback within 60 seconds of submission under a concurrent load of 180 or more students. The system shall process the Celery grading queue without degradation during peak submission periods.

NFR-3 (Scalability): The system shall be deployable on standard institutional hardware without dependency on external cloud services. The system shall support horizontal scaling of the Celery worker pool to handle increased submission volumes across multiple courses simultaneously.

NFR-4 (Reliability): The system shall maintain consistent rubric-based grading output achieving inter-rater reliability above 85 percent across all submission types. The system shall log all grading decisions and override events to a tamper-evident audit trail.

NFR-5 (Maintainability): The system shall be fully containerised using Docker Compose so that the entire platform can be deployed or updated with a single command. The system shall include a GitHub Actions CI/CD pipeline that runs automated tests on every code commit before deployment.

9. System Limitations/Constraints

LI-1: The AI grading engine evaluates only digital text, multiple choice, and code submissions – physical handwritten assignments are outside scope.

LI-2: Grading quality depends entirely on how specific and complete the professor's rubric is; vague rubrics produce weaker results.

LI-3: Smart Proctoring only detects focus loss on the monitored device; it cannot catch a second physical device.

LI-4: The code compiler supports the 12 listed languages domain-specific or non-standard runtimes are not included in this release.

LI-6: The AI Teaching Assistant is limited to uploaded course materials and base model knowledge.

LI-7: LLM fine-tuning is excluded due to hardware and timeline limits.

LI-8: The initial release supports English-language submissions only.

LI-9: Offline mobile functionality is limited to viewing cached grades.

LI-10: No direct ERP integration data import is CSV only.

10. Data Gathering Approach

We conducted semi-structured interviews with six professors, two teaching assistants, and four students to map real grading workflows and pain points. A structured questionnaire went to 120

students (87 usable responses) covering feedback delays and willingness to accept AI grading. We also ran a full observational shadow session with one professor during a live 160-student grading cycle to get exact timing data. Finally, we reviewed 16 peer-reviewed papers on automated essay scoring, explainable AI in education to ground our technical choices.

11. Tools and Technologies

Table 2: Tools and Technologies for Proposed Project

Category	Tool	Version	Rationale
Frontend	go_router	Latest	URL-safe navigation for Flutter web with browser history support
Frontend	Dart	3.x	Strongly-typed language natively built into Flutter SDK
Frontend	fl_chart	Latest	Native Flutter charting library for analytics dashboards
Frontend	Flutter SDK	3.22+	Single codebase for iOS, Android, and Web
Backend	Python FastAPI	0.111+	Async API framework with automatic OpenAPI
Backend	SQLAlchemy	2.x	Async ORM with Alembic migration support
Backend	Celery + Redis	5.x / 7.x	Distributed async task queue for AI grading pipeline
AI / ML	Llama 3.1 8B Instruct	3.1	Open-source model for local grading
AI / ML	Ollama	Latest	Local LLM inference server
AI / ML	LangChain	0.2+	RAG pipeline for Teaching Assistant
AI / ML	LIME	0.2+	Local model-agnostic post-hoc text explanations

AI / ML	SHAP	0.45+	Shapley value-based criterion-level attributions
Code Execution	Judge0 API	v1	Multi-language compilation
Code Execution	re_editor	Latest	Native Flutter code editor with syntax highlighting and Judge0 integration
Database	PostgreSQL	15+	ACID-compliant relational database
Database	Redis	7.2+	Celery broker and explanation cache
DevOps	GitHub Actions	Latest	CI/CD pipeline for automated testing
DevOps	Docker Compose	Latest	Multi-container deployment
Security	JWT / python-jose	3.x	Stateless authentication token management
Design	Figma	Latest	UI/UX wireframing and component design

12. Project Stakeholders and Roles

Table 3: Project Stakeholders for Proposed Project

Stakeholder	Role and Responsibilities
Project Sponsor	Shifa Tameer-e-Millat University (STMU), Islamabad, Department of Software Engineering. Provides academic oversight, infrastructure access for user testing, and formal FYP evaluation.
Primary Supervisor	Mam Habib-un-Nisa. Guides research direction, reviews deliverables, provides domain expertise in software engineering and AI application, and formally approves milestone submissions.

Co-Supervisor	Mr. Muhammad Shaban Qabil. Provides technical guidance on AI/ML architecture and reviews the explainability pipeline design.
Development Team	Muhammad Yasin Asif (BSSE-23S-0036) AI/ML Engine, Backend Architecture, DevOps. Muhammad Fahim (BSSE-23S-0033), Flutter Web Development, Analytics Dashboard. Rayyan Shahid (BSSE-23S-0037), Flutter Mobile Application, Proctoring Module, Code Editor Integration.
End Users	University Professors: primary users of grading, rubric, analytics, and override modules. Students: users of submission, feedback, chatbot, and mobile proctoring modules. Teaching Assistants: secondary graders using collaboration and calibration modules. University Administrators: users of admin dashboard and institutional reporting features.
FYP Evaluation Committee	STMU Department of Software Engineering FYP Committee. Evaluates the project against academic criteria including technical innovation, system completeness, and presentation quality.

13. Module Based Work Division

Table 4: Team Member Work Division for Proposed Project

Student Name	Registration No.	Responsibility / Module / Feature
Muhammad Yasin Asif	BSSE-23S-0036	Module 3 (AI Grading and Evaluation), Module 4 (XAI Processing Engine). Backend API architecture using FastAPI and SQLAlchemy. Celery and Redis task queue configuration.
Muhammad Fahim	BSSE-23S-0033	Module 2 (Course and Assignment Management), Module 6 (Analytics Dashboard), Module 1 partial (Admin user management). Flutter web development, fl_chart data visualization, and PostgreSQL database schema design.

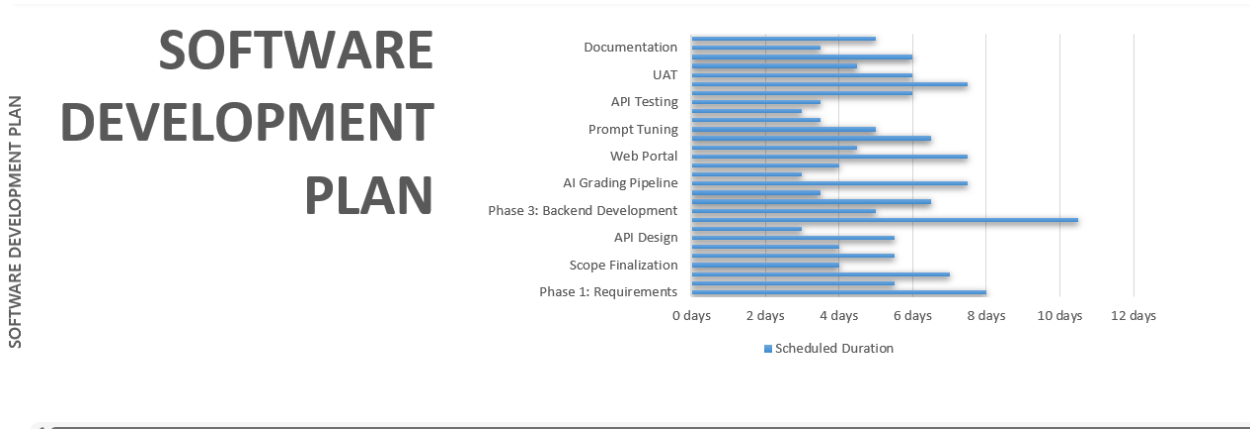
Rayyan Shahid	BSSE-23S-0037	Module 1 partial (Authentication), Module 5 (Student Submission Portal), Module 8 (Code Editor and Compiler), Module 7 (Smart Proctoring), Module 9 (AI Teaching Assistant). Flutter mobile application, Firebase integration, Judge0 API, re_editor, and WebSocket real-time features.
---------------	---------------	---

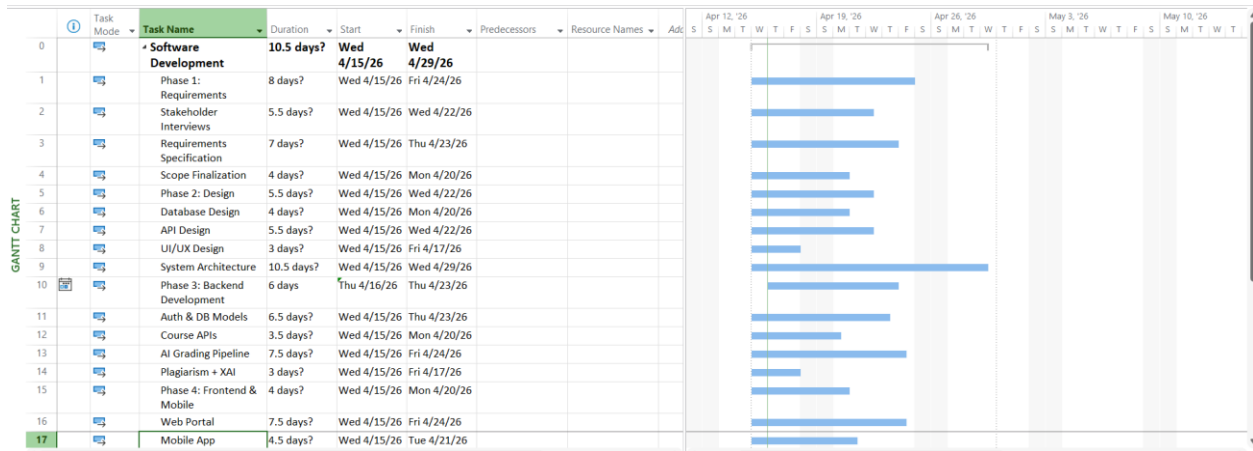
14. WBS and Gantt Chart

The Work Breakdown Structure for ProfessorOS is organized into 7 top-level phases expanded to Level 2 task decomposition.

Estimated durations align with the STMU FYP-I (Semester 7) and FYP-II (Semester 8) schedule, with FYP-I beginning in April 2026 and FYP-II concluding in February 2027.

Phase 3 (Backend) cannot begin until Phase 2 (Design) is complete. Phase 4 (Frontend/Mobile) runs in parallel with Phase 3 after Week 2 of backend development. Phase 6 (Testing) follows completion of Phases 3, 4, and 5.





15. Mockups

Mockup 1: Course and Assignment Rubric Builder

This screen shows a two-panel assignment creation wizard, as illustrated in Figure 1. The left panel contains assignment metadata fields for title, submission type, deadline, maximum marks, and late penalty rules. The right panel displays the Rubric Builder with an Add Criterion button that appends rows containing a criterion name field, four level descriptor inputs labeled Excellent, Satisfactory, Developing, and Insufficient, and a point weight slider. A Preview Assignment button launches a student-view simulation overlay showing exactly how the assignment will appear to students.

Assignment Architect

DashboardCoursesLibrary

New Assignment

DRAFT V1.0

GENERAL INFO

RUBRIC

SETTINGS

ANALYTICS

HISTORY

1 Assignment Details

ASSIGNMENT TITLE

e.g., Comparative Literature Thesis

SUBMISSION TYPE

PDF Upload

MAXIMUM MARKS

100

DEADLINE

mm/dd/yyyy

LATE PENALTY RULES

Deduct % per day

10

Max penalty cap

50

Organize your grading flow with precise rubrics.

Rubric Builder

+ ADD CRITERION

Critical Analysis

CRITERION NAME

WEIGHT

40%

EXCELLENT

Shows profound understanding of the topic with exceptional evidence-based arguments.

SATISFACTORY

Demonstrates a clear understanding of the material with consistent analysis.

DEVELOPING

Grasps core concepts but arguments lack sufficient development or evidence.

INSUFFICIENT

Analysis is superficial or missing entirely with frequent errors in logic.

Structure & Formatting

CRITERION NAME

WEIGHT

25%

EXCELLENT

Impeccable academic formatting with logical, fluid transitions between sections.

SATISFACTORY

Well-structured with clear introduction, body, and conclusion. Minor errors.

DEVELOPING

Follows basic structure but flow is occasionally interrupted by poor organization.

INSUFFICIENT

Lacks clear structure. Formatting requirements are largely ignored.

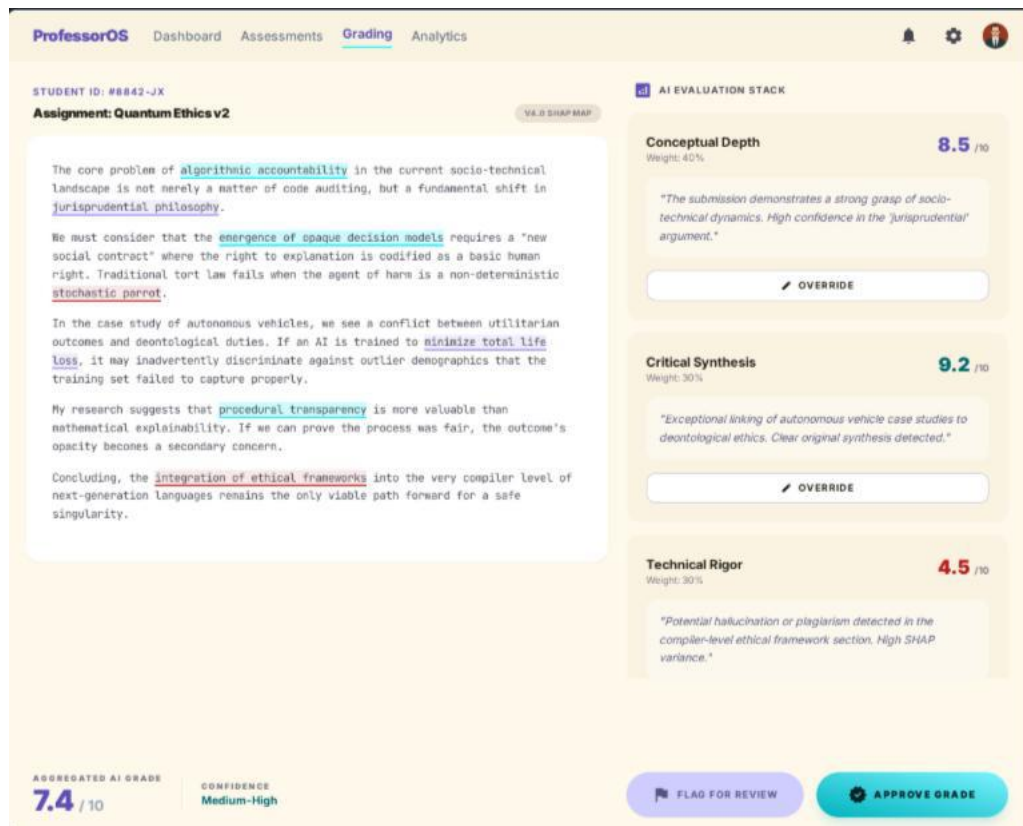
PREVIEW ASSIGNMENT

SAVE DRAFT

NEXT: REVIEW

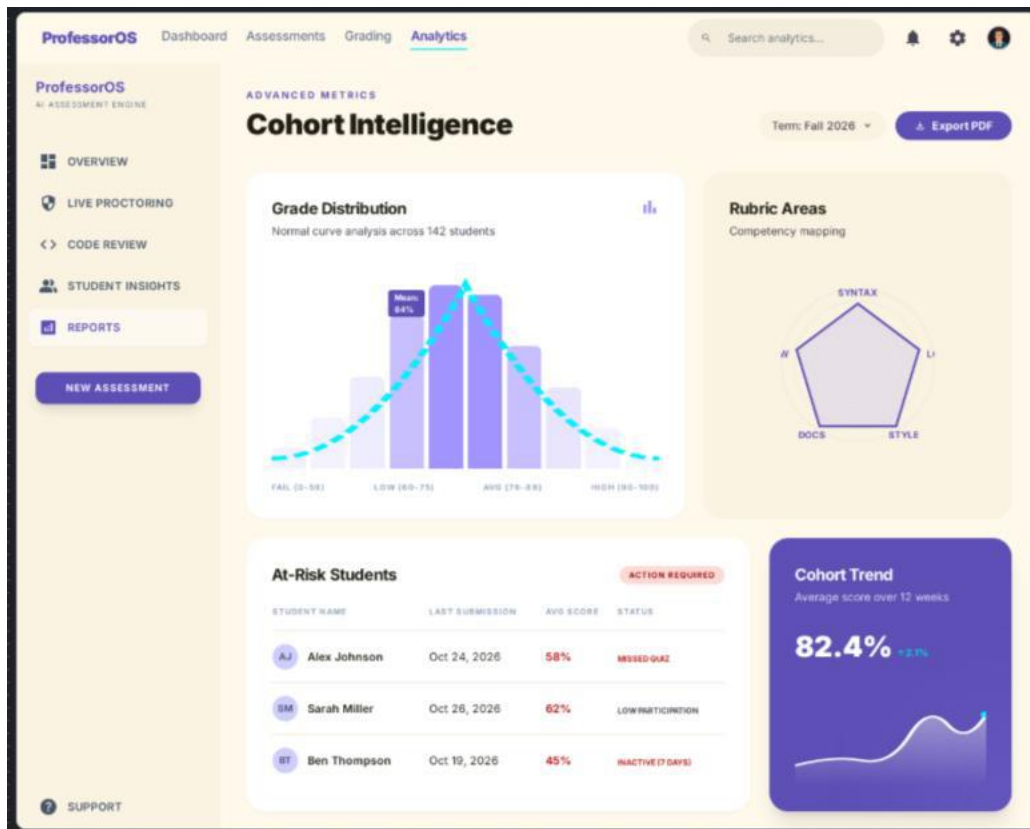
Mockup 2: AI Grading Review Panel

A split-panel interface showing the professor grading view, as illustrated in Figure 2. The left panel displays the full student submission with a SHAP highlight overlay where text spans are colored from green to red based on their contribution to each criterion score. The right panel shows criterion-level cards, each containing the criterion name, AI score, a natural language AI feedback block, and an inline Override button. A total score summary bar and Approve Grade and Flag for Review buttons appear at the bottom.



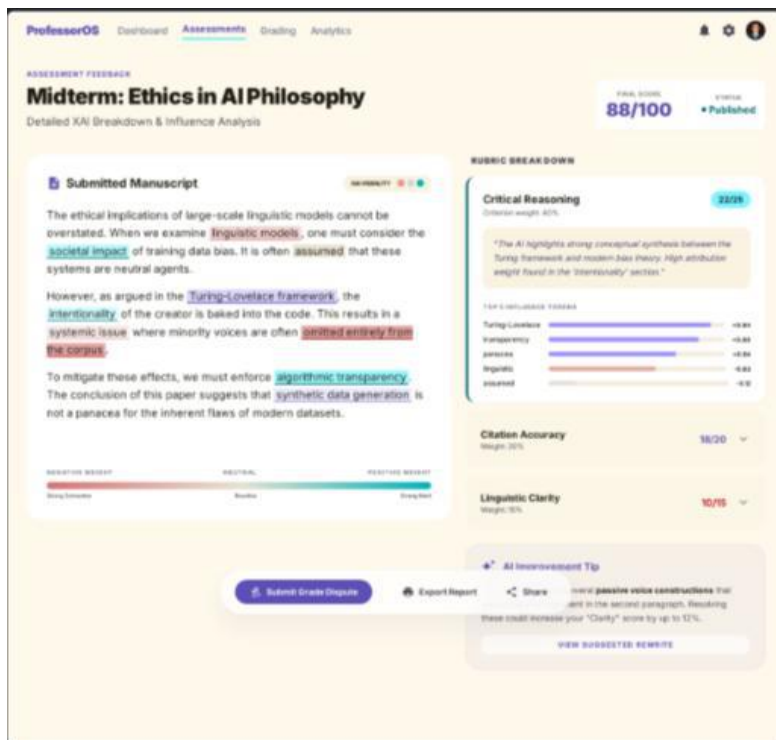
Mockup 3: Analytics Dashboard

A data dashboard with four visualization panels. The top-left panel shows a grade distribution histogram with a normal distribution curve overlay. The top-right panel shows a rubric criterion performance radar chart displaying class-average scores per criterion. The bottom-left panel shows a table of at-risk students listed with their last submission date and current average score. The bottom-right panel shows a time-series line chart of cohort average scores across all assignments in the semester.



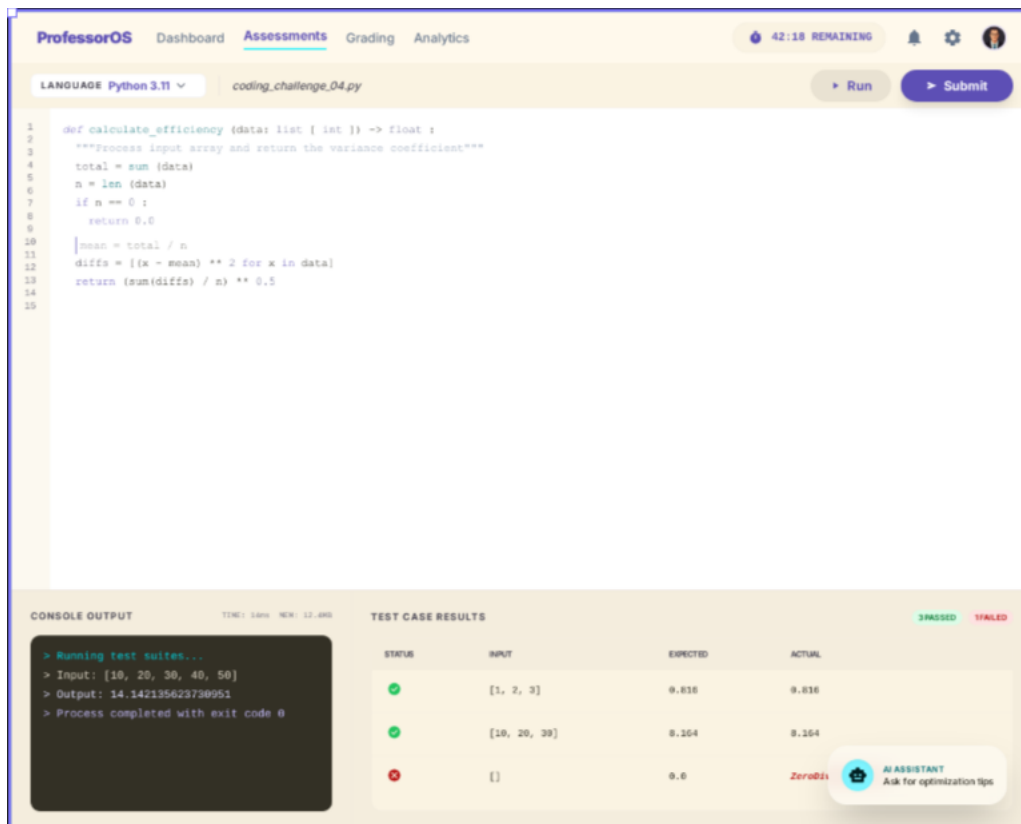
Mockup 4: Student Grade Feedback Viewer with XAI

The student-facing feedback view shows the submitted text on the left with LIME color-coded highlight spans and a legend explaining the color scale from strongly negative to strongly positive influence. The right side shows each rubric criterion as an accordion card containing the criterion name, earned score, AI feedback text, and a bar chart of the top five tokens that most influenced that criterion score. A Submit Grade Dispute button appears at the bottom.



Mockup 5: Integrated Code Editor and Compiler

A full-screen code editor interface built on `re_editor`. The top toolbar contains a language selector dropdown, a Run button, a Submit button, and a time remaining indicator. The main panel `re_editor` renders the code with syntax highlighting. Below the editor is a split output panel where the left side shows console output with execution time and memory usage, and the right side shows test case results as a table with pass or fail icons, input values, expected output, and actual output columns.



Mockup 6: Mobile Smart Proctoring Focus Mode

The Flutter mobile screen during an active proctored examination displays the question interface with a prominent Focus Mode Active banner showing violation count and remaining time. When the student switches applications, a full-screen overlay appears immediately with a red countdown timer and a Return Now button. The background behind the overlay is blurred to prevent content visibility.



Return to Exam Immediately

App-switching detected. Your session will be automatically terminated and flagged for academic dishonesty if you do not return.



[Return Now](#) →

INCIDENT ID: 403-892-FL

16. References

- [1] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier," in Proc. ACM SIGKDD, 2016, pp. 1135-1144.
- [2] D. Hu, "SHAP: A Game Theoretic Approach to Explain the Output of any Machine Learning Model," Advances in Neural Information Processing Systems, vol. 30, 2017.
- [3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP, 2019, pp. 3982-3992.
- [4] H. D. Shermis and J. Burstein, Handbook of Automated Essay Evaluation. New York: Routledge, 2013.
- [5] Meta AI, "Llama 3 Technical Report," Meta Platforms Inc., 2024. [Online]. Available: <https://ai.meta.com/research/publications/>
- [6] A. Ramesh and P. Pradeep, "Automated Assessment in Education: A Literature Review," IEEE Transactions on Learning Technologies, vol. 15, no. 3, pp. 301-315, 2022.
- [7] Judge0, "Judge0 CE: Open-Source Online Code Execution System," GitHub, 2024. [Online]. Available: <https://github.com/judge0/judge0>
- [8] Higher Education Commission (HEC) Pakistan, "Annual Report 2022-2023: Higher Education Statistics," HEC, Islamabad, 2023.
- [9] W. Wang and D. Scott, "Automatic Essay Scoring: A Survey," Natural Language Engineering, vol. 22, no. 4, pp. 537-571, 2016.
- [10] F. Chollet, "Flutter: Beautiful native apps in record time," Google Developers, 2023. [Online].
- [13] R. K. Hambleton and H. Swaminathan, Item Response Theory: Principles and Applications. Boston: Kluwer-Nijhoff Publishing, 1985.

[14] Ollama, "Ollama: Get up and running with large language models locally," GitHub, 2024.
[Online]. Available: <https://github.com/ollama/ollama>

[15] D. B. Payne and J. G. Smith, "Feedback timing and student learning outcomes in higher education," Journal of Educational Research, vol. 48, no. 2, pp. 112-128, 2021.

[16] P. Deane, "On the relation between automated essay scoring and modern views of the writing construct," Assessing Writing, vol. 18, no. 1, pp. 7-24, 2013.

17. Plagiarism Report

Attach the Plagiarism report of your project scope document from library staff of Turnitin tool (<http://turnitin.com>).

Role	Name	Signatures
Supervisor	Ms. Habib Un Nisa	
Co-Supervisor	Mr. Muhammad Shaban Qabil	